

資料結構作業一

1121420 張淙瑜

1. 程式執行平台：

- i. 作業系統：Windows 11
- ii. 編譯器：

```
PS D:\Data_Structure_Assignment\s1121420_HW1> g++ --version
g++.exe (MinGW.org GCC-6.3.0-1) 6.3.0
```

- iii. 開發環境：VS code

2. 如何編譯程式：

- i. 開啟終端機，將路徑切換至程式碼所在資料夾
- ii. 輸入編譯指令：`g++ .\1121420.cpp`
- iii. 執行程式：`.\a.exe`

3. 執行結果截圖：

```
PS D:\Data_Structure_Assignment\s1121420_HW1> g++ .\1121420.cpp
PS D:\Data_Structure_Assignment\s1121420_HW1> .\a.exe
20
20
30
False

3
4
4
2
2
1
True
100
100
True

True
77
77
True
50
40
30
70
70
60
30
False
20
10
10
True
```

4. 實作下 push 與 pop 操作的時間複雜度：

i. Push 操作： $O(n)$ ：

Stack 內已有 n 個元素時，為了讓新加入的元素排在最前面（模擬 Stack 的 Top），必須先把新元素放入空的佇列 $q2$ 中，接著把主要佇列 $q1$ 中的 n 個元素全部 dequeue 出來，再 enqueue 進 $q2$ 裡。最後交換 $q1$ 與 $q2$ 的指標。

ii. Pop 操作： $O(1)$

因為在 push 時已經付出了搬移元素的代價，確保最新加入的元素永遠位在主要佇列 $q1$ 的隊首。所以當執行 pop 時，只需要直接將 $q1$ 執行一次 dequeue 即可取出目標元素，不需要搬移其他資料。

5. 實作下 push 與 pop 操作的時間複雜度（一佇列來模擬堆疊）

i. Push 操作： $O(n)$ ：

當佇列中已經有 n 個元素時（也就是程式碼中的 size），會先將新元素加入隊尾，接著執行一個跑 n 次的 for 迴圈。這個迴圈會將原本的 n 個舊元素逐一 dequeue 取出，再立刻 enqueue 放回隊尾。因為這個「旋轉」操作的執行次數與堆疊內現有的元素數量 n 成正比，所以時間複雜度為線性時間 $O(n)$ 。

ii. Pop 操作： $O(1)$ ：

每次 push 時，都已經透過上述的旋轉操作把最新加入的元素推到了佇列的最前面（隊首），所以當需要 pop 時，只需要直接呼叫 dequeue()，將隊首元素移除並回傳即可。這個動作不需要再搬移任何資料，所以時間複雜度為常數時間 $O(1)$ 。

6. 程式流程圖

